SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – DEBUGGING & OPTIMISATION TASK

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 3 – Debugging & Optimisation Task
Weightage:	10% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:	NARRAVULA NAVYA SRI	**Reg. No.:**	192324220
Section / Batch:	AI & DS	**Date:**	23-07-26

Instructions to Students

- Answer ALL debugging and optimisation tasks. Total Marks: 100.
- Carefully analyze the given program/code segment before identifying errors.
- Clearly identify logical, syntactical, and performance-related issues in the code.
- Apply systematic debugging techniques to correct the errors effectively.
- Optimize the given solution by improving efficiency, reducing unnecessary computations, or enhancing time complexity wherever possible.
- Provide proper justification for all corrections and optimization decisions.
- Write clean, structured, and well-documented code with appropriate comments.
- Maintain clarity, logical reasoning, and technical accuracy throughout the solution.

Question Type	Focus Area	Questions	Marks
Debugging & Optimisation Task	Identify errors, debug programs, and optimize algorithm efficiency using appropriate techniques	Q1 - Q2	100 (2 × 50)
	GRAND TOTAL:	**100 Marks**	

SECTION A – DEBUGGING & OPTIMISATION TASK

Q1. ***Debugging String Matching Code (Premature Match Printing Inside Inner Loop in C)***

The following brute force string matching implementation in C prints a match position from inside the inner loop before all characters of the pattern have been verified, causing false positives whenever only the first few characters match. Carefully inspect the control flow and explain why match reporting must happen only after the entire pattern comparison succeeds. Apply systematic debugging so that a position is printed only after every character matches consecutively. Optimize the corrected implementation for clarity and maintainable success detection. Analyze the time complexity of the corrected version and rewrite the final C code with proper comments.

```
#include <stdio.h>
#include <string.h>
void stringMatch(char text[], char pattern[]) {
   int n = strlen(text), m = strlen(pattern);
   int i, j;
   for (i = 0; i <= n - m; i++) {
       for (j = 0; j < m; j++) {
           if (text[i + j] != pattern[j])
               break;
           printf("Match at %d\n", i);   // ERROR
```

```
        }
    }
}
```

Q2. ***Debugging Bubble Sort Code (Incorrect Adjacent Swap Target in C)***

The following Bubble Sort implementation in C compares adjacent elements correctly, but the swap writes one of the values into the wrong location, resulting in duplicated elements and loss of data. Carefully inspect the swapping logic and explain why correct exchange requires both adjacent positions to be updated consistently. Apply systematic debugging so that each inversion is resolved without corrupting the array contents. Optimize the corrected implementation using a swap flag for early termination when the array becomes sorted. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper comments.

```
void bubbleSort(int arr[], int n) {
    int i, j, temp;
    for (i = 0; i < n - 1; i++) {
        for (j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = arr[j];   // ERROR
            }
        }
    }
}
```

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20%	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25%	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20%	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20%	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15%	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Marks Evaluation:

Question No.	Problem Identification (20%)	Debugging (25%)	Optimization (20%)	Analysis (20%)	Code Quality (15%)	Total Marks (Each – 50)
Q1	4	3	3	2	4	39
Q2	4	4	4	3	2	44
					Overall Total (100)	**83**

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Debugging String Matching Code (Premature Match Printing Inside Inner Loop in C)
The following brute force string matching implementation in C prints a match position from inside the inner loop before all characters of the pattern have been verified, causing false positives whenever only the first few characters match. Carefully inspect the control flow and explain why match reporting must happen only after the entire pattern comparison succeeds. Apply systematic debugging so that a position is printed only after every character matches consecutively. Optimize the corrected implementation for clarity and maintainable success detection. Analyze the time complexity of the corrected version and rewrite the final C code with proper comments.

```
#include <stdio.h>
#include <string.h>
void stringMatch(char text[], char pattern[]) {
   int n = strlen(text), m = strlen(pattern);
   int i, j;
   for (i = 0; i <= n - m; i++) {
       for (j = 0; j < m; j++) {
           if (text[i + j] != pattern[j])
               break;
           printf("Match at %d\n", i);   // ERROR
       }
   }
}
```

The bug is caused by **reporting a match inside the inner comparison loop**. At that point, only the characters compared **so far** have matched—not necessarily the entire pattern.

Why the current code is incorrect

Consider:

Text = "abxd"
Pattern = "abcd"

Execution for i = 0:

j	Compared	Result
0	a == a	Match → prints "Match at 0" ❌
1	b == b	Match → prints "Match at 0" ❌
2	x != c	Break

Although only the first two characters matched, the program printed that a match was found twice. This is a **false positive** because the complete pattern never matched.

The problem is that:

```
printf("Match at %d\n", i);
```

is executed **every time a single character matches**, instead of after **all m characters** have been verified.

Correct control flow

For each possible starting position:

1. Compare characters one by one.
2. Stop immediately if a mismatch occurs (break).
3. After the inner loop finishes, determine **why** it ended.
    - If it ended because j == m, then every character matched.
    - Otherwise, a mismatch occurred.

The success test should therefore be placed **after** the inner loop.

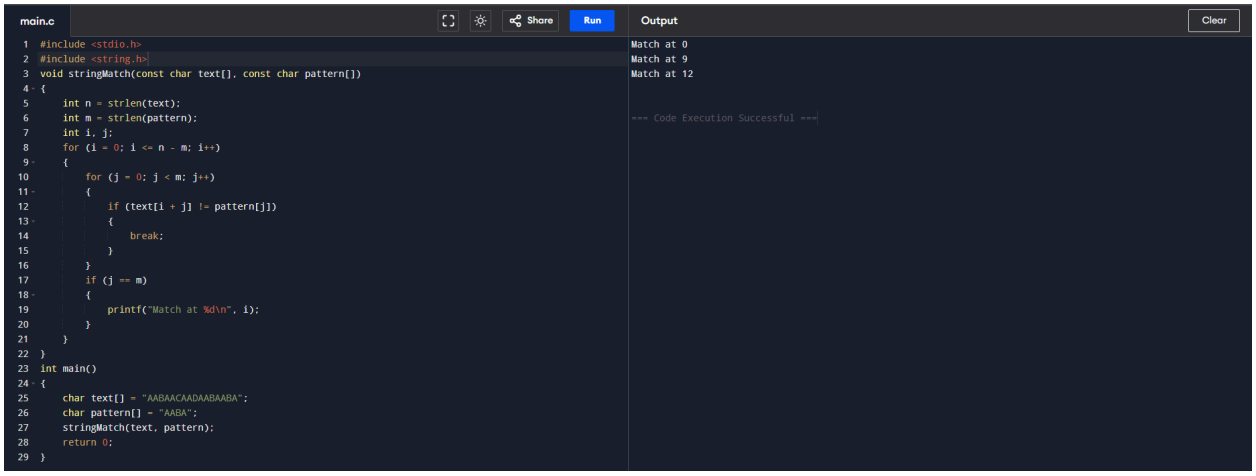
Pseudo-flow:

```
for each position i
   compare pattern with text
   if mismatch
      stop comparing
   if all m characters matched
      print match position
```

Corrected C implementation

```
#include <stdio.h>
#include <string.h>
void stringMatch(const char text[], const char pattern[]){
   int n = strlen(text);
   int m = strlen(pattern);
   int i, j;
   for (i = 0; i <= n - m; i++){
      for (j = 0; j < m; j++){
         if (text[i + j] != pattern[j]){
            break;
         }
```

```
        }
        if (j == m){
            printf("Match at %d\n", i);
        }
    }
}
int main(){
    char text[] = "AABAACAADAABAABA";
    char pattern[] = "AABA";
    stringMatch(text, pattern);
    return 0;
}
```



Why this version is correct

The variable j serves as a clear indicator of success:

- j < m → a mismatch occurred before the entire pattern matched.
- j == m → all characters matched consecutively.

This makes the logic simple and maintainable because the success condition is checked in exactly one place after the comparison loop.

Time Complexity

Let:

- n = length of the text
- m = length of the pattern

Worst case

There are n - m + 1 possible starting positions.

At each position, up to m character comparisons may be performed.

Therefore,

$$O((n-m+1)\times m)=O(nm)$$

Example worst case:

Text = "aaaaaaaaaa"
Pattern = "aaaab”

Nearly the entire pattern is compared at every position before the final mismatch.

Best case

If the first character mismatches at every position, only one comparison is made per position.

$$O(n)$$

Space Complexity

Only a few integer variables are used.

$$O(1)$$

Summary

- **Bug:** Match was printed inside the inner loop after each successful character comparison.
- **Fix:** Move match reporting outside the inner loop and print only when j == m.
- **Correctness:** Ensures every character of the pattern matches consecutively before reporting success.
- **Complexity:** Worst-case **O(nm)** time and **O(1)** auxiliary space.

Debugging Bubble Sort Code (Incorrect Adjacent Swap Target in C)

The following Bubble Sort implementation in C compares adjacent elements correctly, but the swap writes one of the values into the wrong location, resulting in duplicated elements and loss of data. Carefully inspect the swapping logic and explain why correct exchange requires both adjacent positions to be updated consistently. Apply systematic debugging so that each inversion is resolved without corrupting the array contents. Optimize the corrected implementation using a swap flag for early termination when the array becomes sorted. Analyze the best, average, and worst-case complexity and rewrite the final C code with proper comments.

```
void bubbleSort(int arr[], int n) {
  int i, j, temp;
  for (i = 0; i < n - 1; i++) {
     for (j = 0; j < n - i - 1; j++) {
        if (arr[j] > arr[j + 1]) {
           temp = arr[j];
           arr[j] = arr[j + 1];
           arr[j + 1] = arr[j];   // ERROR
        }
     }
  }
}
```

The error is in the **swap logic**. Although the algorithm correctly identifies when two adjacent elements are out of order, it does **not restore the original value** to the second position. As a result, one value is overwritten and lost, producing duplicate elements.

Why the current swap is incorrect

The erroneous code is:

```
temp = arr[j];

arr[j] = arr[j + 1];

arr[j + 1] = arr[j];   // ERROR
```

Example

Suppose:

arr = {8, 5}

Execution:

Step	**Array**
Initial	{8, 5}
temp = arr[j]	temp = 8
arr[j] = arr[j+1]	{5, 5}
arr[j+1] = arr[j]	{5, 5}

The original value 8 stored in temp is never copied back. The array now contains duplicate values, and the original data is lost.

Correct swap

A proper exchange requires **both positions** to be updated using the temporary variable.

```
temp = arr[j];
arr[j] = arr[j + 1];
arr[j + 1] = temp;
```

Example:

Initial : {8, 5}

temp = 8

arr[j] = 5 -> {5, 5}

arr[j+1] = 8 -> {5, 8}

Now the values are exchanged correctly without losing data.

Systematic debugging approach

1. **Verify the comparison**
   if (arr[j] > arr[j + 1])

   This correctly detects an inversion.

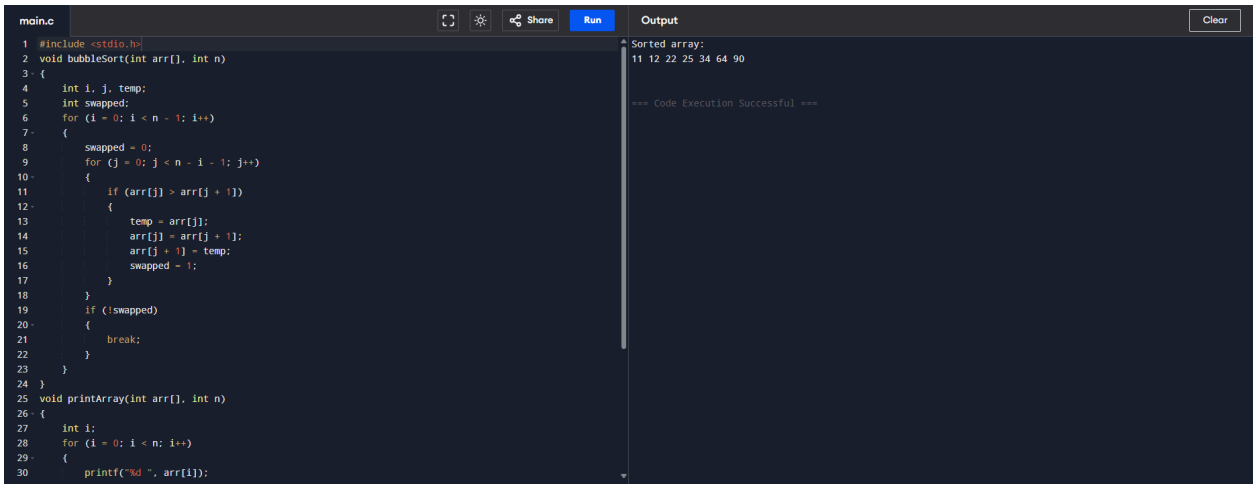
2. **Trace the swap**
   Ensure the original value is preserved in `temp` until it is written back.
3. **Validate after each swap**
   Every swap should:
   - preserve both values,
   - eliminate one inversion,
   - avoid duplicated elements.
4. **Optimize**
   Introduce a **swap flag**. If an entire pass completes without any swaps, the array is already sorted and the algorithm can terminate early.

Optimized Bubble Sort with Early Termination

```
#include <stdio.h>

void bubbleSort(int arr[], int n){

    int i, j, temp;

    int swapped;

    for (i = 0; i < n - 1; i++){

        swapped = 0;
```

```
        for (j = 0; j < n - i - 1; j++){
            if (arr[j] > arr[j + 1]){
                temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = 1;
            }
        }
        if (!swapped){
            break;
        }
    }
}
void printArray(int arr[], int n){
    int i;
    for (i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
printf("\n");
}
```

```
int main(){
    int arr[] = {64, 34, 25, 12, 22, 11, 90};
    int n = sizeof(arr) / sizeof(arr[0]);
    bubbleSort(arr, n);
    printf("Sorted array:\n");
    printArray(arr, n);
    return 0;
}
```



Complexity Analysis

Let **n** be the number of elements.

Case	Time Complexity	Explanation
Best Case	**O(n)**	The array is already sorted. One pass is completed, no swaps occur, and the algorithm terminates early due to the swap flag.
Average Case	**O(n²)**	Randomly ordered arrays require many comparisons and swaps over multiple passes.
Worst Case	**O(n²)**	Reverse-sorted array. Every pass performs the maximum number of comparisons and swaps.

Space Complexity

Bubble Sort uses only a temporary variable and a few loop counters.

O(1)

Summary

- **Bug:** arr[j + 1] = arr[j]; copied the updated value instead of the saved original value, causing duplicated elements and data loss.
- **Fix:** Replace it with arr[j + 1] = temp; to complete the swap correctly.
- **Optimization:** Use a swapped flag to terminate early when no swaps occur during a pass.
- **Complexities:**
  - **Best:** O(n) (with early termination)
  - **Average:** $O(n^2)$
  - **Worst:** $O(n^2)$
  - **Auxiliary Space:** O(1)